BrokerChain: A Blockchain Sharding Protocol by Exploiting Broker Accounts

Huawei Huang, *Senior Member, IEEE*, Zhaokang Yin, *Graduate Student Member, IEEE*, Qinde Chen, *Graduate Student Member, IEEE*, Jian Zheng, Xiaofei Luo, Guang Ye, *Graduate Student Member, IEEE*, Xiaowen Peng, Zibin Zheng, *Fellow, IEEE*, and Song Guo, *Fellow, IEEE*

***Abstract*—State-of-the-art blockchain sharding solutions, such as Monoxide, can cause severely imbalanced distribution of transaction (TX) workloads across all blockchain shards due to the deployment policy of their accounts. Imbalanced TX distributions then produce *hot shards*, in which the cross-shard TXs may experience an unlimited confirmation latency. Thus, how to address the hot-shard issue and how to reduce cross-shard TXs become significant challenges of blockchain sharding. Through reviewing the related studies, we find that a cross-shard TX protocol that can achieve workload balance among all shards and simultaneously reduce the quantity of cross-shard TXs is still absent from the literature. To this end, we propose BrokerChain, which is a cross-shard blockchain protocol dedicated to account-based state sharding. Essentially, BrokerChain exploits fine-grained state partition and account segmentation. We also elaborate on how BrokerChain handles cross-shard TXs through broker accounts. The security issues and other properties of BrokerChain are analyzed rigorously. Finally, we conduct comprehensive evaluations using an open-source blockchain sharding prototype named *BlockEmulator*. The evaluation results show that BrokerChain outperforms other baselines in terms of transaction throughput, transaction confirmation latency, the queue size of the transaction pool, and workload balance.**

***Index Terms*—Blockchain, sharding protocol, blockchain sharding, blockchain systems.**

I. Introduction

THE sharding technique is viewed as a promising solution that can improve the scalability of blockchains [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]. The idea of sharding is to *divide and conquer* when dealing with all the transactions (TXs) submitted to a blockchain system. Instead of processing all TXs by all blockchain nodes following the conventional manner, the sharding technique divides blockchain nodes into smaller committees, each only having to handle a subset of TXs [2]. Thus, the transaction throughput can be improved exponentially.

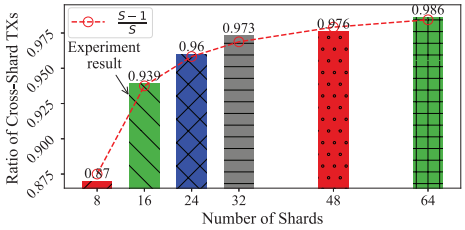


Fig. 1. Ratio of cross-shard transactions *vs* the number of shards (i.e., *S*). The experimental results of the CTX ratio closely match the theoretical analysis.

There are mainly three types of sharding paradigms, i.e., network sharding, transaction sharding, and state sharding. Since *network sharding* divides the entire blockchain network into smaller committees, it is viewed as the foundation of the other two sharding paradigms. In each round of the conventional sharding protocol, after the committee's formation, several disjoint subsets of TXs are assigned to those committees following the paradigm of *transaction sharding*. Then, committees run a specific consensus protocol locally, such as Practical Byzantine Fault Tolerance (PBFT) [12], to achieve a local consensus towards the set of assigned TXs. Among those three sharding techniques, *state sharding* is the most difficult one because it has to ensure that all states of a blockchain are amortized by all shards. Currently, the state sharding is mainly staying in the early-stage theoretical study. Several representative sharding solutions have been proposed, such as Elastico [2], Omniledger [3], RapidChain [5], and Monoxide [9]. Those sharding solutions are based on either *UTXO* (Unspent Transaction Output) or *account/balance* transaction model. For example, Elastico [2] exploits the UTXO model, while Monoxide [9] adopts the account/balance transaction model.

Motivation. The performance of existing sharding systems is mainly constrained by two factors. Firstly, the blockchain-sharding system has a high ratio of cross-shard transactions (abbr. as CTXs) when the number of shards grows. Secondly, the workloads across different shards show a severely unbalanced distribution.

Received 5 February 2024; revised 29 January 2025; accepted 3 March 2025; approved by IEEE TRANSACTIONS ON NETWORKING Editor S. Schmid. Date of publication 26 March 2025; date of current version 20 August 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB2702304; in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2025B1515020053; in part by the National Natural Science Foundation of China (NSFC)/Research Grants Council (RGC) Collaborative Research Scheme under Grant 62461160332 and Grant CRS_HKUST602/24; in part by NSFC under Grant 62272496; and in part by the Fundamental Research Funds for the Central Universities, Sun Yat-sen University, under Grant 23lgbj019. The conference version [DOI: 10.1109/INFOCOM48880.2022.9796859] of this article was presented in INFOCOM 2022. *(Corresponding author: Song Guo.)*

Huawei Huang, Zhaokang Yin, Qinde Chen, Jian Zheng, Xiaofei Luo, Guang Ye, Xiaowen Peng, and Zibin Zheng are with the School of Software Engineering, Sun Yat-sen University, Zhuhai 519082, China, and also with Guangdong Engineering Technology Research Center of Blockchain, Guangzhou 230000, China (e-mail: huanghw28@mail.sysu.edu.cn).

Song Guo is with the Department of Computer Science and Engineering (CSE), The Hong Kong University of Science and Technology (HKUST), Hong Kong, SAR (e-mail: songguo@cse.ust.hk).

Digital Object Identifier 10.1109/TON.2025.3550502

2998-4157 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.

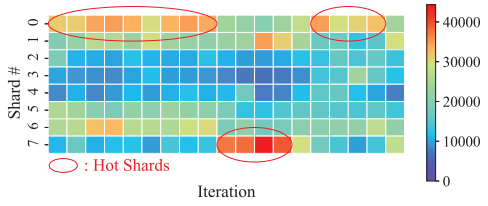


Fig. 2. When feeding 80000 TXs for each iteration of blockchain consensus, Monoxide's sharding protocol [9] yields imbalanced transaction distributions among all shards. Consequently, some *hot shards* are induced. Here, we call a shard congested by an overwhelming number of TXs a *hot shard*.

Using Fig. 1, let's explain how the number of shards affects the cross-shard transaction ratio. Given $S \in \mathbb{N}^+$ as the number of shards and assuming that accounts are randomly allocated to different shards, the probability that the payer account and the payee account of a transaction are not in the same shard is $\frac{S-1}{S}$. That is, the probability of a transaction becoming a CTX is $\frac{S-1}{S}$. To investigate the impact of the number of shards on the CTX ratio, we conduct motivating experiments using the historical Ethereum transaction data. The number of shards varies from 8 to 64. In Fig. 1, the dashed line shows the *theoretical analysis* between the CTX ratio and the number of shards if adopting the random account allocation method. Meanwhile, the histograms show the actual CTX ratios observed while invoking Monoxide [9] in our experiments. Fig. 1 shows that the CTX ratio yielded by experiments closely matches the theoretical results. When the number of shards is larger than 32, the ratio of cross-shard transactions exceeds 97%. In Monoxide, the atomic execution of each CTX is guaranteed through a *transaction relay* mechanism [9], in which a CTX is divided into a *deduction* and a *deposit* sub-transactions. The blockchain-sharding system processes them in the payer's shard and payee's shard of this CTX, respectively. However, this method increases the total transaction processing workload drastically.

In account-based state sharding, user accounts are distributed to different blockchain shards according to the first few bits of their addresses, aiming to store all account states collaboratively. Although this manner can improve the system throughput, CTXs are inevitably induced. The handling of CTXs is a primary technique issue in the account/balance-based blockchain sharding system. Monoxide [9] guarantees the atomicity of CTXs by introducing relay TXs. However, relay TXs may result in congested shards. We call such shards congested by an overwhelming number of TXs the *hot shards*. In each hot shard, TXs cannot be processed in time and thus may experience long confirmation latency. To prove our concern about Monoxide's sharding protocol, we evaluate the TX workloads of all shards by feeding 80000 TXs at each iteration of Monoxide's consensus. Fig. 2 shows the TX distributions of Monoxide's shards. We observe that hot shards widely exist in the system across multiple iterations. These congested hot shards are caused by active accounts, which launch a large number of TXs frequently in their associated shards. Consequently, TXs are distributed over all shards in an imbalanced way. The insight behind those observations is that the account-deployment method adopted by Monoxide does not consider the frequency of launching TXs by accounts. Moreover, a great number of CTXs could be also caused by Monoxide's sharding protocol. As a result, when the number of shards becomes large, almost all TXs are CTXs. When the payee account of a CTX is suspended in a congested hot shard, Monoxide may incur infinite confirmation latency for the transactions residing in local shards' transaction pools.

Furthermore, through a thorough review of the state-of-the-art studies such as Elastico [2], Omniledger [3], and RapidChain [5], we find that most of the cross-shard TX mechanisms mainly focus on the UTXO transaction model. Although Monoxide [9] and SkyChain [13] design their sharding protocols by adopting the account/balance model, the hot-shard issue has not been addressed. In summary, we have not yet found a cross-shard TX solution that can eliminate hot shards for the account/balance-based state sharding. Thus, we are motivated to devise such a new sharding protocol.

Challenges. When assigning a large number of TXs in the account/balance-based state sharding, a natural problem is how to reach workload balance across all shards. Recall that the imbalanced workloads in hot shards can cause large TX confirmation latency, which threatens the eventual atomicity of CTXs. This problem becomes even more challenging when most of the TXs are cross-shard ones [9]. Therefore, guaranteeing the eventual atomicity of CTXs becomes a technical challenge that prevents the state-sharding mechanism from being largely adopted in practice.

To this end, this paper proposes a cross-shard blockchain protocol, named *BrokerChain*, for account/balance-based state sharding. BrokerChain aims to reduce the number of cross-shard TXs and ensure workload balance for all blockchain shards at the same time.

The proposed BrokerChain protocol relies on the key role named as *broker accounts*, which are some voluntary accounts that are willing to act as the intermediaries to help handle cross-shard TXs. We call such intermediaries the *broker accounts*, which are shortened as *brokers* hereafter. We use the accounts and the ledger shown in Fig. 3 as an example to demonstrate how a broker handles a cross-shard TX. Suppose that C is a broker, whose account is segmented and stored in shards #1 and #2. Now, we have a raw CTX, i.e., payer A transfers a number v of tokens to the payee account B via broker C. Here, we call shard #1 the *source shard* and shard #2 the *destination shard* of this CTX. Through the broker C, the raw CTX can be transformed into two intra-shard TXs, which shall be dealt with in the *source shard* and *destination shard*, respectively. Therefore, the broker accounts are the fundamental components of the proposed BrokerChain protocol.

Our study in this paper leads to the following **contributions**.

- **Originality.** BrokerChain partitions the account-state graph and performs account segmentation in the protocol layer. Thus, the well-partitioned account states can be amortized by multiple shards to achieve workload balance across all shards.
- **Methodology.** To alleviate the hot-shard issue, BrokerChain includes a cross-shard TX handling mechanism, which can guarantee the *duration-limited* eventual atomicity of CTXs.
- **Usefulness.** We implemented BrokerChain and deployed a prototype in Alibaba Cloud. The transaction-driven experimental results proved that BrokerChain outperforms state-of-the-art baselines in terms of throughput, TX confirmation latency, the queue size of transaction pools, and workload balance.

The rest of this paper is organized as follows. Section II reviews state-of-the-art studies. Section III describes the

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.

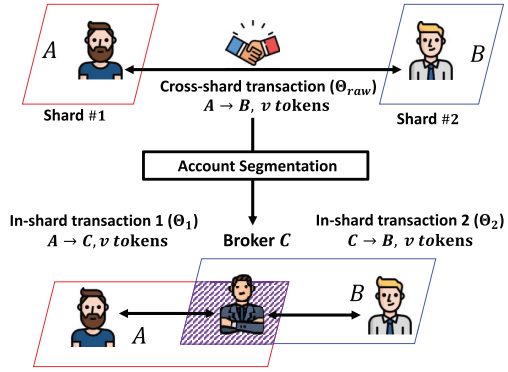


Fig. 3. How a broker account turns a CTX into two intra-shard TXs. Here, the *account segmentation* is described in Section III-C.

protocol design. Section IV depicts the handling of CTXs. Section V analyzes the security issues and other properties of BrokerChain. Section VII demonstrates the performance evaluation results. Finally, Section VIII concludes this paper.

II. Preliminaries and Related Work

A. Transaction Models

There are two mainstream transaction models in existing blockchain systems, i.e., the UTXO model [14] and the account/balance model [15]. Under the UTXO model, a TX may involve multiple inputs and outputs. When the outputs originating from previous UTXOs are imported into a TX, those UTXOs will be marked as *spent* and new UTXOs will be produced by this TX. Under the account/balance model, users may generate a TX using the deposits in their accounts. Ethereum [15] employs the account/balance model due to its simplicity in that each TX has only one payer account and one payee account. When confirming the legitimacy of a TX, it is necessary to check whether the deposits of the payer's account are sufficient or not.

B. Representative Blockchain Sharding Solutions

A great number of sharding solutions have been proposed to improve the scalability of blockchain systems [16]. Some of these representative solutions are reviewed as follows. Elastico [2] is viewed as the first sharding-based blockchain system, in which every shard processes TXs in parallel. Kokoris et al. [3] then present a state-sharding protocol named OmniLedger, which adopts a scalable BFT-based consensus algorithm to improve TX throughput. Facing the high overhead of sharding reconfiguration in Elastico and OmniLedger, Zamani et al. [5] propose RapidChain to solve this overhead issue. Chainspace [4] exploits a particular distributed atomic commit protocol to support the sharding mechanism for smart contracts. Then, Wang et al. [9] propose Monoxide, which realizes an account/balance-based sharding blockchain. In Monoxide, a novel relay transaction mechanism is leveraged to process cross-shard TXs. Next, Nguyen et al. [6] present a new shard placement scheme named OptChain. This method can minimize the number of cross-shard TXs for the UTXO-based sharding. Prism [7] achieves optimal network throughput by deconstructing the blockchain structure into atomic functionalities. Dang et al. [8] present a scaling sharded blockchain that can improve the performance of consensus and shard formation. Recently, Tao et al. [10] propose a dynamic sharding system to improve the system throughput based on smart contracts. Huang et al. [11] propose an online stochastic-exploration algorithm to schedule the most valuable committees for the large-scale sharding blockchain.

C. Cross-Shard Transaction Processing

A sharding blockchain must consider how to handle the cross-shard TXs. In Omniledger [3], the authors adopt a client-driven two-phase commit (2PC) mechanism with lock/unlock operations to ensure the atomicity of cross-shard TXs. Differently, Chainspace [4] introduces a client-driven BFT-based mechanism. RapidChain [5] transfers all involved UTXOs to the same shard by sub-transactions, such that a cross-shard TX can be transformed to intra-shard TXs. In Monoxide [9], the deduction operations and deposit operations are separated. Relay transactions are then exploited to achieve the eventual atomicity of cross-shard TXs. Recently, Pyramid [17] introduces a layered sharding consensus protocol. In this solution, cross-shard TXs are handled by a special type of nodes that serve two shards at the same time. Mizrahi et al. [18], [19] investigate how to find an efficient sharding approach by considering the transaction processing cost and the memory for mapping accounts to shards simultaneously.

Compared with these existing studies, the proposed BrokerChain handles cross-shard TXs by taking advantage of broker accounts. Furthermore, BrokerChain can achieve a workload balance while distributing TXs through the fine-grained state-partition and account-segmentation mechanisms.

III. Network and Protocol Design of BrokerChain

In this section, we present BrokerChain, an adaptive state-sharding protocol based on the account/balance transaction model.

A. Overview of BrokerChain

Similar to Rapidchain [5], we first define an *epoch* as a fixed length of the system running time in BrokerChain. To avoid Sybil attacks [20], blockchain nodes should get their identities by solving a hash-based puzzle before joining in a network shard. In such puzzle, an unpredictable common randomness is created at the end of the previous epoch. Once a blockchain node solves the puzzle successfully, the last few bits of the solution indicate which shard the node should be designated to. In our design, BrokerChain consists of two types of shards.

- **M-shard.** A mining shard (shorten as *M-shard*) generates TX blocks by packing TXs and achieves the intra-shard consensus at the beginning of each epoch.
- **P-shard.** A partition shard (shorten as *P-shard*) is devised to partition account states in an adaptive manner during each epoch.

BrokerChain requires a number $S \in \mathbb{N}^+$ of M-shards and one P-shard existing in the sharding blockchain. For both M-shard and P-shard, we adopt PBFT protocol [12] to achieve their respective intra-shard consensus and to avoid blockchain forks. Using Fig. 4, we now describe the most important four sequential phases of BrokerChain as follows.

- **Phase ①: TX-Block Consensus.** At the beginning of an epoch, each M-shard packages TXs from the TX pool, and generates several TX blocks by running PBFT protocol. The number of TX blocks generated by an M-shard in an epoch depends on the intra-shard network parameters such as network bandwidth. Note that, the

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.

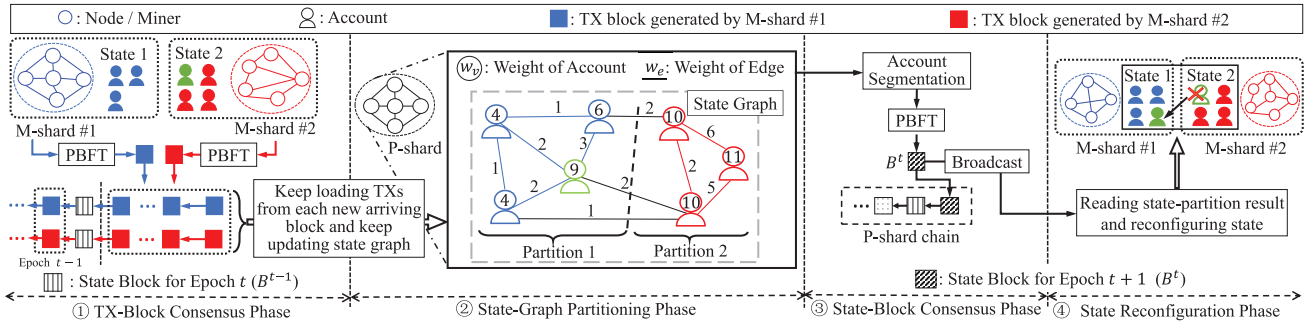


Fig. 4. Four major phases of BrokerChain protocol in an epoch t. There are two types of shards in BrokerChain, i.e., the partition shard (P-shard) and mining shards (M-shard). Here, we only use two M-shards to illustrate how BrokerChain protocol works.

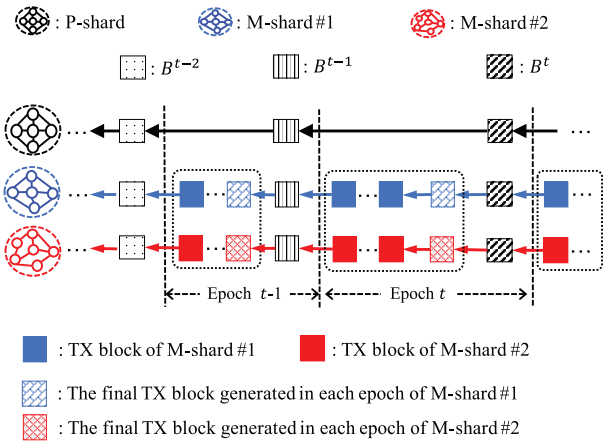


Fig. 5. Illustration of how blocks connect between epochs (e.g., B^{t-1} is a state block generated in epoch t-1 but followed by the first transaction blocks of M-shards in epoch t).

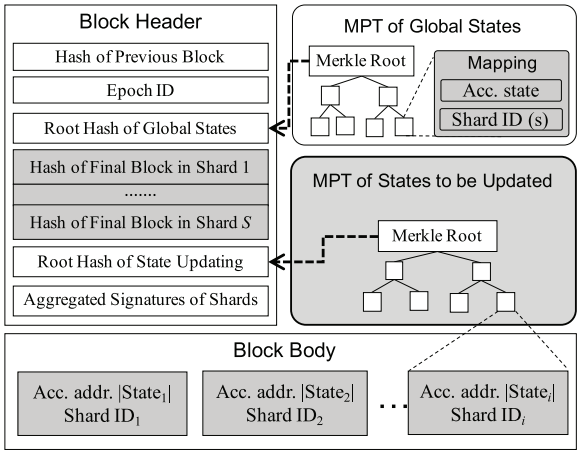


Fig. 6. Data structure design of the *state block*. The modules with grey background are designed particularly for state updating.

first new TX block generated by an M-shard follows the *state block* (i.e., B^{t-1} shown in Fig. 4), which records the state-partition results of the previous epoch. Fig. 5 illustrates how state blocks connect transaction blocks.

- **Phase ②: State-Graph Partitioning.** P-shard keeps loading the TXs from the new arriving blocks generated by M-shards, and keeps updating the state graph of all accounts. Once all TX blocks have been generated by M-shards within the current epoch, the state graph is fixed. Then, the protocol begins to partition the state graph, aiming to achieve workload balance among all shards.
- **Phase ③: State-Block Consensus**. With the state graph of all accounts partitioned in the previous phase, BrokerChain performs the *account segmentation*, which is described in detail in Section III-C. To reach a consensus towards the result of both state-graph partition and account segmentation, PBFT protocol is exploited again to generate a *state block* (denoted by B^t), which is then added to the P-shard chain.
- **Phase ④: State Reconfiguration**. After reaching a consensus on the partition result, the P-shard broadcasts the state block B^t, which contains the S-partitioned new state graph, to all the associated M-shards. When receiving a state block B^t, an M-shard reads the state-partition result from the state block, and reconfigures its states accordingly such that TXs in the next epoch t+1 can be distributed to the designated shards according to the new account states.

At the end of each epoch, BrokerChain also needs to update the formation of both P-shard and M-shards. To update those shards, the Cuckoo rule [21] is invoked such that the system can defend against the join-leave attacks [22], [23].

In the following, we elaborate on the most critical operations and data structures in BrokerChain, i.e., *State-Graph Partition*, *Account Segmentation*, and the *modified Shard State Tree*.

B. State-Graph Partition

Updating State Graph. P-shard monitors the TX blocks generated by M-shards, and keeps loading the TXs in each arriving new block in real time to build/update the *state graph*. When receiving a specified minimum number of TX blocks, the state graph is fixed. As shown in the State-Graph Partitioning Phase of Fig. 4, each vertex of such state graph represents an account. The edge weight (denoted by w_e) is defined as the number of TXs associated with the corresponding pair of accounts, while the vertex weight (denoted by w_v) is calculated by the sum of the associated edges' weight. Given the fixed state graph, P-shard begins to partition all accounts in the graph by invoking Metis [24], which is a well-known heuristic graph-partitioning algorithm. Metis can partition the state graph into non-overlapping $S \in \mathbb{N}^+$ sectors, while reducing the number of cross-shard TXs and considering the workload balance across shards.

Designing the Data Structure for State Block. At epoch t, a *state block* B^t is generated by the P-shard in Phase ③ *State-Block Consensus*. After that, as shown in Phase ④ *State Reconfiguration*, B^t shall be broadcast to all

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.

M-shards for updating account states in each M-shard. Fig. 6 depicts the data structure of the state block, say B^t. The block header forms a chain structure by containing the hash of the previous state block, ensuring the immutability of the P-shard chain. The block header also records the hash of the *final transaction block* produced by each M-shard in the current epoch t. This design enables state block B^t to follow the final block generated by M-shard at each epoch t. Global states and the state updating information of the accounts that need to be migrated are both stored using Merkle Patricia Trie (MPT). The hashes of their Merkle roots, i.e., `Root Hash of Global States` and `Root Hash of State Updating`, are also stored in the block header. Using these two root hashes, M-shards can verify and track account migration. The block body stores the mapping between account states and shard IDs. Such the design of the block body enables state updates for M-shards in the next epoch $t+1$.

C. Account Segmentation

First, the *account segmentation* is a mechanism of BrokerChain that can allow a specific broker account to be split into multiple smaller sub-accounts, but they share the same account address. Using such the *account segmentation* mechanism, a broker can help turn a CTX into two intra-shard TXs, just as shown in Fig. 3.

In Monoxide [9], an account must be stored in only one single shard. The fact is that an account, such as an exchange's active account, may participate in a great number of TXs. This kind of active account would result in hot shards inevitably. Regarding the hot-shard issue, we give an example in the following to show that it could be solved at the user layer. The only required assumption is that a user is allowed to hold multiple accounts, which are then permitted to be stored in multiple arbitrary state shards. Suppose that a specific user is holding multiple accounts. Following Monoxide's account-deployment rule, this user's accounts will be placed in different shards if the first $k \in \mathbb{N}^+$ bits of account addresses are different. Then, this user can launch a large number of TXs through his multiple accounts such that those TXs can be distributed into designated shards. Consequently, the workload balance of all shards is possible to achieve, and the number of cross-shard TXs can be reduced as well, simultaneously.

Motivation to Account Segmentation. Users, however, generally do not have the motivation to create multiple accounts and deposit their tokens there. Because it's inconvenient to manage those distributed multiple accounts. What's more, it does not make sense to force users to launch their TXs through their multiple accounts, aiming to balance the overall workload of a blockchain system. Therefore, we propose an *account segmentation* mechanism, which works in the protocol layer of blockchain architecture. The proposed account segmentation mechanism has the following advantages. i) Users are not forced to open multiple accounts. ii) A user account's state can be easily divided and stored in multiple shards. iii) Through this user-transparent way, the workload balance of all shards is convenient to achieve such that hot shards can be addressed.

Example of Account Segmentation. To better understand the proposed account segmentation mechanism, we use Fig. 7 to illustrate an example. Suppose that we have a sharding blockchain system with two shards (shard #1 and #2) and three accounts (denoted by *A*, *B*, and *C*). Now the ledger includes

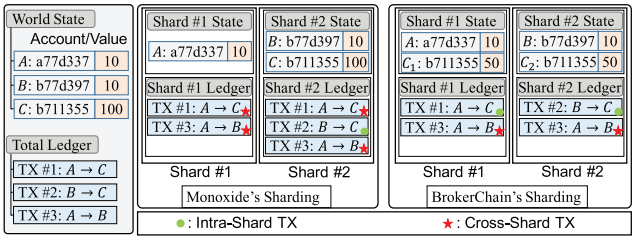


Fig. 7. Comparison of account segmentation results between Monoxide and BrokerChain. We see that BrokerChain yields fewer cross-shard TXs and a more balanced TX workload across all shards.

3 original transactions `TX #1:A→C`, `TX #2:B→C`, and `TX #3:A→B`. If the sharding system has already deployed accounts *A* and *B* in shard #1 and #2, respectively. Then account *C* can only be stored in shard #2 (or shard #1) under Monoxide's policy. As a result, the TXs "`TX #3:A→B`" and "`TX #1:A→C`" become 4 cross-shard TXs. In contrast, under BrokerChain, account *C* (with 100 tokens) can be segmented into two smaller accounts C_1 and C_2 (50 tokens for each), which are then placed to both shards, respectively. This account segmentation result is equivalent to that account *C* has 50 tokens stored in shard #1, and another 50 tokens stored in shard #2. Thus, the original TX "`TX #1:A→C`" turns to an intra-shard TXs. Furthermore, the workloads of both shards are balanced perfectly. From the example illustrated above, we see that the imbalanced TX workloads can be much alleviated and the number of cross-shard TXs can be reduced under BrokerChain.

The difference between the proposed account segmentation and the case where users autonomously store their deposits in multiple smaller accounts is that the addresses of the account segmented by BrokerChain are all identical. That is, BrokerChain protocol stores the deposits of an account located at different shards with the same account address. To identify an account's multiple states located at different shards, we adopt Ethereum's *counter* mechanism [15], which is called the *nonce* of an account's state.

D. Modified Shard State Tree (mSST)

To enable the *state-graph partitioning* and *account segmentation* operations, BrokerChain has to enforce each shard to know the *storage map* of all accounts. Therefore, we devise a modified Shard State Tree (mSST) to store the account states based on Ethereum's *world state tree* [15].

In contrast to the original world state tree, BrokerChain's mSST is built on top of the storage map of all accounts. We denote such storage map as a vector $\mathbf{\Psi} = [e_1, e_2, \ldots, e_S]$ with each element $e_i = 0/1$, $i \in \{1, 2, \ldots, S\}$, where $S \in \mathbb{N}^+$ is the number of M-shards. Note that, BrokerChain needs to configure a storage map for every account. Only when e_i is equal to 1, can this specific account be viewed as being stored in shard *i*. If there are multiple elements in $\mathbf{\Psi}$ that are labeled to 1, then the associated account is segmented to the same number of accounts, which are stored in each specific shard, respectively.

Data Structure of mSST. Fig. 8 illustrates the data structure design of mSST. The account state $\mathbb{S}_\mu$ of a specific user μ is represented by:

$$\mathbb{S}_\mu = \{X_\mu \mid \mathbf{\Psi}, \eta, \omega, \zeta\},$$

where X_μ denotes the account address of user μ, and η denotes the *nonce* field, which indicates the number of TXs sent from

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.

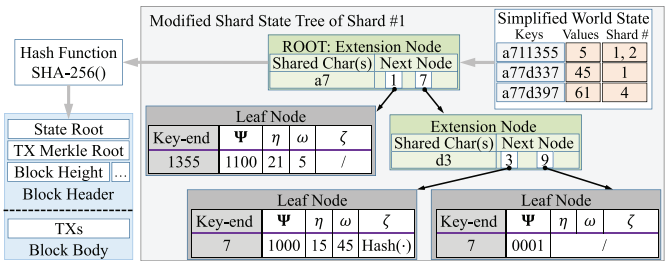
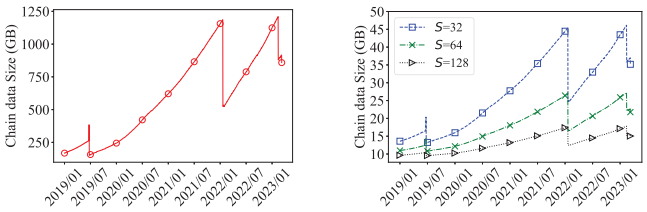


Fig. 8. Data structure of the modified *Shard State Tree* (mSST).



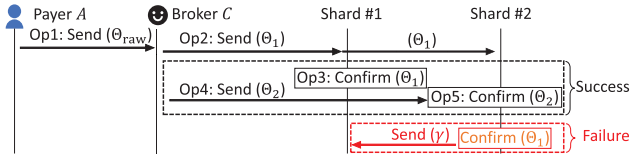
(a) Ethereum storage (b) Sharding system's storage

Fig. 9. Comparison of the storage overhead between Ethereum and the sharding system.

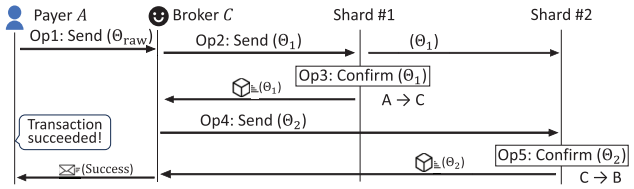
address X_μ or the contract creation operation generated by user μ. Then, ω denotes the *value* field, which shows the token deposits of the user. Finally, ζ denotes the *code* field, which represents the account type. Here, the account types include the user account and the smart contract account (the hash of the smart contract). To a specific account, different shards maintain different mSSTs for this target account. The difference is determined by the *value*, *nonce* and *code* fields of the local mSST. If a target account is not stored in a shard anymore, those fields corresponding to this account in the shard's mSST will be removed. And this shard needs to only update the states for the target account. Any change of the target account's state will cause the change of state root in mSST. Therefore, it is easy to maintain the consistency of an account's state in the associated shard.

Query Complexity. Through querying the *storage map* in mSST, we can easily confirm whether a TX is an intra-shard TX or a cross-shard one. The querying time order is $O(1)$. Therefore, the protocol can easily get the deposits of an account by summing the *value* field of all its segmented account states. The deposit-querying time order is $O(\xi)$, where ξ is the number of shards where the account's segmented states are stored.

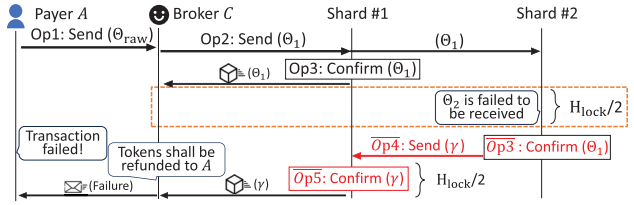
Storage Complexity. By March 2023, the number of accounts in Ethereum has exceeded 224 million. The full node needs to store approximately 858 Gigabytes (GB) of on-chain data. As shown in Fig. 9(a), even though Ethereum periodically prunes the world state tree, the full nodes still need to maintain high storage capacity. Thus, traditional single-chain structures cannot meet the needs of large-scale applications since ordinary nodes have to address the challenge of storage if they choose to serve as consensus nodes. However, in a sharding system, full nodes in blockchain shards only need to store a small portion of state data compared with that of Ethereum full nodes. Fig. 9(b) shows that the chain data size declines following the increasing number of shards. Thus, the storage complexity of the proposed protocol has the potential to serve large-scale blockchain applications.



(a) The overall workflow of processing a CTX.



(b) Successful handling of a CTX.



(c) The handling of a CTX under the failed case.

Fig. 10. The success and failure cases during the handling of CTXs. A success case includes all five operations: Op1-Op5, while a failure case typically indicates that Θ_2 is not received by shard #2 within shard #1's block-height interval $[H_{\text{current}}, H_{\text{current}} + H_{\text{lock}}/2]$.

IV. Handling the Cross-Shard Transactions

As depicted in the previous section, the *account segmentation* mechanism enables an account to be segmented into multiple smaller ones. That is why a single broker can have multiple *sub-account states* located in different shards. In this section, we elaborate on how BrokerChain handles cross-shard TXs through brokers based on the *account segmentation* mechanism. A fundamental requirement to become a broker is that its account has a sufficient amount of tokens. For the system users who intend to become brokers, they could request to pledge their assets to a trusted third party or a special smart contract. Then, the BrokerChain protocol accommodates the broker eligibility for those interested system users. To stimulate a system user to play a broker's role, an incentive mechanism is necessary. In fact, we have proposed such an incentive mechanism called Broker2Earn [25], which is actually a DeFi (decentralized finance) protocol built on top of BrokerChain. However, other versions of the incentive mechanism are possibly designed to recruit brokers for BrokerChain. We encourage readers who are interested in our BrokerChain to develop their incentive mechanisms.

We still use the accounts and the ledger shown in Fig. 3 to demonstrate the handling of cross-shard TXs. When running the proposed protocol, as shown in Fig. 10(a), BrokerChain processes each CTX in either a successful or failed case. Typically, a failed case is caused by large network latency while confirming the relay sub-transactions. By exploiting Fig. 10, we introduce both the success and failure cases of handling cross-shard TXs.

A. Atomicity Guarantee in A Success Case

Fig. 10(b) demonstrates the five operations of the successful handling of a CTX. The five operations are described as follows.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.

- **Op1: Create a raw transaction** Θ_{raw}. Account A first chooses an appropriate token-lock duration (denoted by H_{lock}), which is in fact the number of successive blocks counting from the block including Θ_{raw} to the one releasing locked tokens. Θ_{raw} is defined as follows.

$$\Theta_{\text{raw}} := \langle\langle B, v, C, H_{\text{lock}}, \eta_{\text{payer}}, \eta_{\text{broker}}\rangle, \sigma_A\rangle,$$

where η_{broker} and η_{payer} denote the nonce of broker C and the nonce of payer account A, respectively. Symbol σ_A is the signature of payer A, calculated by invoking ECDSA algorithm [26]. The usage of the token-lock duration H_{lock} and nonces $\eta_{\text{payer}}, \eta_{\text{broker}}$ are explained subsequently. Next, A informs Θ_{raw} to the broker C.
- **Op2: Create the first-half cross-shard TX** Θ_1. When receiving Θ_{raw}, broker C creates the first-half cross-shard TX, denoted by Θ_1. Then, the current block height in shard #1 when Θ_1 is created is labeled as H_{current}. We then have

$$\Theta_1 := \langle\langle \text{Type1}, \Theta_{\text{raw}}, H_{\text{current}}\rangle, \sigma_C\rangle,$$

where Type1 is an indicator of Θ_1, and σ_C is the signature of broker C. Next, Θ_1 is broadcast to the blockchain network. When all the blockchain nodes in other shards receive Θ_1, they execute the following steps. i) Validating Θ_1 using signatures σ_A and σ_C. ii) Acquiring the payer's public key and calculating the payer's account address using signature σ_A. iii) Through querying the mSST, shard nodes know the source and destination shards. iv) Using Kademlia [27] routing protocol, Θ_1 is forwarded to shards #1 and #2. After validating that η_{payer} is correct and the deposits of payer A are sufficient, Θ_1 will be added to the TX pool of shard #1, and waits to be packaged in a block.
- **Op3: Confirm** Θ_1. After broadcasting, the blockchain nodes in shard #1 shall include Θ_1 in the block whose height is labeled as H_{source}. Apparently, $H_{\text{current}} \leq H_{\text{source}}$. Then, payer A transfers v tokens to broker C, and these tokens will be locked within the block-height interval $[H_{\text{source}}, H_{\text{source}} + H_{\text{lock}}]$. The nonce of payer A will be increased by 1 to prevent the replay attack [28].
- **Op4: Create the second-half cross-shard TX** Θ_2. When broker C is acknowledged that Θ_1 has been confirmed, C then creates the second-half cross-shard TX Θ_2:

$$\Theta_2 := \langle\langle \text{Type2}, \Theta_{\text{raw}}\rangle, \sigma_C\rangle,$$

where Type2 is a label of Θ_2. Next, Θ_2 will be broadcast to the blockchain network. Θ_2 is finally routed to the destination shard, i.e., shard #2. When η_{broker} is verified qualified and the deposits of broker C in the shard #2 are sufficient, Θ_2 will be added to the TX pool of shard #2.
- **Op5: Confirm** Θ_2. When Θ_2 is received by the blockchain nodes in shard #2, it will be packaged in a new block if the current block height of the shard #1 is smaller than $H_{\text{current}} + H_{\text{lock}}/2$. The account state in shard #2 is then updated when the following token transfer is executed: broker C transfers a number v of tokens to payee account B. Broker C's nonce is then increased by 1 to prevent the replay attack [28].

In the *success* case of CTX handling, the payee account B can get a number v of tokens when Θ_2 is successfully included in the destination shard's block. On the other hand, only when the block height of shard #1 is greater than $H_{\text{source}} + H_{\text{lock}}$, can broker C receive a number of v refunded tokens.

B. Atomicity Guarantee in A Failure Case

According to our design shown in Fig. 8, all shards mutually share their block headers where the essential block height information is included. Thus, if the blockchain nodes in shard #2 fail to receive Θ_2 when the current block height of shard #1 exceeds $H_{\text{current}} + H_{\text{lock}}/2$, we say that Θ_2 is failed to be acknowledged by shard #2. This failure result is very possibly induced by a malicious broker who intends to embezzle the payer account's locked tokens.

As shown in Fig. 10(c), in order to process such a failure case, BrokerChain executes the following steps to guarantee the atomicity of CTXs.

- **Op3: Confirm** Θ_1. When Θ_2 is failed, Θ_1 will be included in a block located at shard #2. Broker C's nonce will be attempted to update in shard #2.
- **Op4: Send** γ. When Θ_1 is confirmed, blockchain nodes in shard #2 send the following confirmation of Θ_1 to shard #1 as the *failure proof* (denoted by γ):

$$\gamma := \langle \Theta_1, \textit{dest}, H_{\text{dest}}, \{P_{\text{dest}}\}\rangle,$$

where *dest* represents the index of the destination shard, H_{dest} denotes the height of destination shard's block where Θ_1 is included, and $\{P_{\text{dest}}\}$ refers to the Merkle tree path [14]. $\{P_{\text{dest}}\}$ consists of the hash values of all the associated Merkle tree nodes along the path originating from Merkle tree root and terminating at the entry hash node corresponding to Θ_1. The Merkle tree path $\{P_{\text{dest}}\}$ is used to verify that Θ_1 is packaged in a block located at shard #2.
- **Op5: Confirm** γ. Once the nodes in shard #1 receive the failure proof γ and verify the correctness of path $\{P_{\text{dest}}\}$, shard #1 is acknowledged that Θ_1 has been included in a block located at shard #2 but Θ_2 was not. Then, γ will be included in the source shard's block whose block height is smaller than $H_{\text{source}} + H_{\text{lock}}$. Finally, the locked tokens in Op3 will be refunded to the payer account A.

No matter whether a CTX is handled in a success or failure case, only one of the two created TXs Θ_1 or Θ_2 is allowed to be included in a block at the destination shard. The failure-proof γ could be possibly lost by accident during broadcasting. Once γ is found lost, it can be reconstructed and launched again by the destination shard.

V. Property Analysis of BrokerChain

A. Tackling Double-Spending Attacks

BrokerChain treats the payee account of a cross-shard TX benign. Thus, in the following, we analyze how BrokerChain tackles typical security threats brought by the payer and broker.

Threat-1: Sender A is malicious. A malicious payer account may launch double-spending attacks. In such attacks, payer A may create a double-spending TX in the source shard along with the raw TX Θ_{raw} denoted by $\Theta_A := \langle A \rightarrow A', \eta_A\rangle$, where A' is payer A's another account and η_A is the nonce of Θ_A. Note that, to achieve double spending, η_A must be exactly same with η_{payer} signed in the firsts-half cross-shard TX Θ_1. Although the attack strategy described above seems feasible, it is impossible to implement under BrokerChain. Taking the advantage of the *counter* mechanism aforementioned, if Θ_A is confirmed by the source shard and η_A is updated, then Θ_1 will be failed to be confirmed by the source shard because its Θ_1's nonce η_{payer} is conflict to η_A. Only when broker C finds that Θ_1 has been confirmed by the source shard, C begins to create

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.

Θ_2. Through this way, broker C can defend against double-spending attacks launched by the malicious payer account.

Threat-2: Broker C is malicious. A malicious broker C may create the following double-spending TX in the destination shard exactly when Θ_1 is confirmed in the source shard: $\Theta_C := \langle C \to C', \eta_C \rangle$, where C' is broker's another account and η_C is exactly same with η_{broker} signed in TX Θ_2. Similarly, if Θ_C is confirmed by the destination shard and the nonce of broker C is updated, then Θ_2 will not be confirmed by the destination shard because η_{broker} is conflict. By monitoring the update of nonce η_{broker}, BrokerChain can identify whether a broker is malicious or not. To prevent broker C from embezzling payer A's pledged tokens, BrokerChain enforces the handling routine of such cross-shard TX to fall into the failure case.

In conclusion, with the proposed asset pledge and token-lock mechanisms, BrokerChain can ensure the atomicity of cross-shard TXs under the threats of double-spending attacks.

B. Recommended Setting for Token-Lock Duration

Recall that in Op1 of cross-shard TX's handling described in Section IV-A, a payer needs to first choose an appropriate token-lock duration towards creating a raw transaction. The question is how to set such token-lock duration for the raw transaction. A feasible approach is to set the token-lock duration according to the system throughput observed currently. An empirical recommended setting is to ensure at least 20 times of the average latency of handling the cross-shard TXs.

C. Analysis of Cross-Shard Confirmation

As shown in Fig. 10, the cross-shard verifications occurred during the handling of cross-shard TXs include the following two cases: i) under the success case, broker C has to verify that whether Θ_1 is included in the source shard's chain, and ii) under the failure case, the source shard's nodes need to verify the *failure proof* γ sent by the destination shard. Thus, in the success case, the computing complexity of cross-shard verification is $O(1)$. In the failure case, the computing complexity of cross-shard verification is $O(n)$, where n is the number of blockchain nodes in the source shard. Therefore, BrokerChain can lower such cross-shard verification overhead by shifting the on-chain verification to the off-chain manner taking advantages of the broker's role.

Regarding the cross-shard TX's confirmation latency, it is theoretically longer than twice intra-shard TX's confirmation latency. This is because a cross-shard TX must be confirmed at the source and destination shards in a sequential order. In contrast, BrokerChain can lower the confirmation latency based on broker's reputation. If a broker is completely trusted by the sharding blockchain, both Θ_1 and Θ_2 can be allowed to execute simultaneously. Thus, their confirmation latency can approach to that of an intra-shard TX technically.

D. Duration-Limited Eventual Atomicity

Monoxide [9] guarantees the *eventual atomicity* of cross-shard TXs. However, Monoxide does not specify the confirmation time of relay TXs. Thus, the cross-shard TX's handling in Monoxide may take a long time. In contrast, BrokerChain can ensure each cross-shard TX is done within the predefined token-lock duration H_{lock}. When the cross-shard TX is successfully processed, the payer account's pledged tokens will be received by the payee account. Otherwise, payer's pledged tokens will be refunded by the broker. Therefore, we claim that BrokerChain ensures the *duration-limited* eventual atomicity for cross-shard TXs.

VI. Security Analysis of Shards

Considering malicious consensus nodes exist in a Byzantine-fault network, this section presents the theoretical failure probability of both P-shards and M-shards if they are cracked by malicious nodes. The insights disclosed in this section could offer some risk analysis for developers when they implement a real-world BrokerChain-like blockchain.

A. Security Analysis for P-Shard

1) How to Calculate the Failure Probability of P-Shard: First, we give Theorem 1 to suggest that the blockchain system should not allow nodes to join the P-shard freely without any restrictions.

Let real number-valued symbols $\alpha \in (0,1)$ denote the proportion of all honest nodes' computational power that join in the *P-shard*, $\varphi \in (0,1)$ denote the proportion of malicious nodes' computational power in the *overall system*, and $\upsilon \in (0,1)$ denote the probability that a node is malicious if it is selected randomly from the P-shard.

Theorem 1: Suppose that blockchain nodes can freely choose which shard to join, we have $\upsilon = \frac{\varphi}{\varphi+\alpha\cdot(1-\varphi)}$, *where* $\varphi + \alpha \cdot (1-\varphi) > 0$.

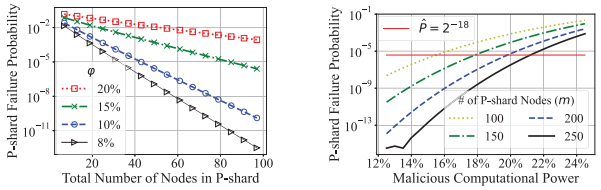
Proof: Nodes join the sharding system by solving hash puzzles, where computational power dominates the security of a shard. To help better understand our calculation, we define the *relative* proportion of computational power of either malicious or honest nodes as their ***credits*** while constructing shards. Because nodes can choose the type of shards to join, malicious nodes could concentrate their computational power to attack the P-shard. We already know that φ is the *credits* of malicious nodes while constructing the P-shard. Thus, the *credits* of honest nodes in the P-shard is $\alpha \cdot (1-\varphi)$. Thereby, the equation of $\upsilon = \frac{\varphi}{\varphi+\alpha\cdot(1-\varphi)}$ holds. The proof of Theorem 1 concludes. □

Remark 1: Theorem 1 tells us that when the malicious nodes' computational power φ is fixed, the risk of suffering from computational-power attacks in the P-shard increases if α declines. When α becomes smaller, the effect of malicious nodes' relative computational power (i.e., the credits of malicious nodes) will be amplified in the P-shard. For example, by fixing φ=20%, when α=0.9, we have υ=21.7%; but when α=0.5, we have υ=33.3%. Therefore, the blockchain system should not allow nodes to join the P-shard freely.

We then let $m \in \mathbb{N}^+$ denote the number of nodes in P-shard, $\vartheta \in \mathbb{N}^+$ denote the number of nodes in each M-shard, $\kappa \in (0,1)$ denote the probability that a node is assigned to P-shard by the blockchain system, and $\hat{P} \in (0,1)$ denote the probability of the P-shard failure. Here, we call a ***P-shard failure*** occurs if the P-shard is cracked by malicious nodes.

When the system randomly assigns nodes to all shards, we have $\upsilon = \varphi$, which indicates that the computational power of malicious nodes will not be amplified as Theorem 1 shows. Via the following Theorem 2, we calculate the probability of the P-shard failure.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.



(a) Failure Probability ($\hat{P}$) *vs* P-shard Size (m)

(b) Failure Probability ($\hat{P}$) *vs* Malicious Computational Power (φ)

Fig. 11. Theoretical failure probability of the *P-shard*.

Theorem 2: Suppose that the blockchain system randomly assigns nodes to all shards. The probability of the P-shard failure is calculated as follows:

$$\hat{P} = \sum_{i=\lfloor (m-1)/3 \rfloor + 1}^{m} \binom{m}{i} (\upsilon)^i \cdot (1-\upsilon)^{m-i}. \tag{1}$$

Proof: Since shard nodes are randomly assigned, the probability that malicious nodes join P-shard depends solely on the proportion of malicious nodes out of the total number of nodes. The total number of nodes is $m+\vartheta \cdot S$, thus $\kappa = \frac{m}{m+\vartheta \cdot S}$, where m and ϑ are the numbers of nodes in the P-shard and each M-shard, respectively. The *credits* of malicious nodes in P-shard is $\varphi \cdot \kappa$ and the *credits* of honest nodes in the P-shard is $(1-\varphi) \cdot \kappa$. Therefore, the equation of $\upsilon = \varphi$ holds, i.e., the impact of malicious nodes' computational power is not amplified. The intra-shard consensus protocol PBFT can tolerate at most 1/3 Byzantine nodes in each shard. When nodes are randomly assigned to different shards, the probability of the P-shard failure $\hat{P}$ is written as Eq. (1). The proof of Theorem 2 concludes. □

2) The Implications of Failure Probability of P-Shards: The failure probability of P-shards $\hat{P}$ given in Theorem 2 can help developers estimate the security level of the blockchain given system parameters m and ϑ.

Theorems 1 and Theorems 2 imply that the system should enforce nodes randomly joining different shards. According to $\hat{P}$, the system should keep an appropriate number of nodes in the P-shard to make the probability of the P-shard failure lower than a certain predefined threshold, e.g., $\hat{P} \leq 2^{-\lambda}$, where $\lambda \in \mathbb{N}^+$ is the parameter that describes the security level of the sharding system. When $\lambda = 18$, the probability of the P-shard failure is not greater than $2^{-18} \approx 3.8 \times 10^{-6}$, which means that a P-shard failure is expected to occur once every 2.6×10^5 times of shard reconfiguration. When the number of the P-shard consists of 250 nodes (i.e., $m = 250$), the P-shard can tolerate malicious nodes whose total computational power reaches 21% at the most.

Fig. 11(a) shows that the probability of the P-shard failure drops following the number of nodes in the P-shard while changing φ from 20% to 8%. A smaller φ results in a lower probability of the P-shard failure. Fig. 11(b) shows that the probability of P-shard failure increases when the malicious nodes' computational power increases. Note that, the red line indicates the security threshold when $\lambda = 18$. While changing the size of the P-shard from 100 to 250, we observe that a large number of nodes can reduce the P-shard failure given a specific φ.

B. Security Analysis for M-Shard

1) How to Calculate the Failure Probability of M-Shards: Let the real number-valued symbol $\delta \in (0,1)$ denote the probability that an M-shard node is malicious; the integer variable $N \in \mathbb{N}^+$ denote the total number of malicious nodes in all M-shards; the symbol `shard i` ($i \in \mathbb{N}^+$) as the M-shard whose ID is $i \in [S]$; the integer $n_i \in \mathbb{N}^+ (i \in [S])$ denote the number of malicious nodes in `shard i`; and the integer $\beta \in \mathbb{N}^+$ denote the threshold for the number of malicious nodes in each M-shard. If n_i exceeds threshold β, the M-shard `i` cannot work properly. We then let the real number-valued symbol $p(n_i)$ denote the probability that `shard i` has a number n_i of malicious nodes, and the real number-valued symbol $\overline{P} \in (0,1)$ denote the probability that an M-shard is failed because of malicious nodes. Finally, the integer $\vartheta \in \mathbb{N}^+$ denotes the number of nodes in any equal-sized M-shard.

Theorem 3: Assuming nodes are randomly assigned to each shard and the system satisfies Byzantine fault tolerance, we have $\beta = \lfloor \frac{\vartheta - 1}{3} \rfloor$, and the probability of M-shard failure (denoted by $\overline{P}$) is calculated as follows:

$$\overline{P} = 1 - \sum_{N=0}^{\vartheta \cdot S} \Big\{ (\varphi)^N (1-\varphi)^{\vartheta \cdot S - N} \times \sum_{n_1=0}^{\min\{N,\beta\}} \cdots \sum_{n_S=0}^{\min\left\{N - \sum_{j=1}^{S-1} n_j, \beta\right\}} \binom{\vartheta}{n_1} \cdots \binom{\vartheta}{n_S} \Big\}. \tag{2}$$

Proof: We first introduce a random integer variable $X \in \mathbb{N}^+$ that represents the number of malicious nodes in all M-shards. Because the nodes are randomly assigned to shards, X follows the binomial distribution. Thus, we have

$$p(X = N) = \binom{\vartheta \cdot S}{N} \cdot (\delta)^N (1-\delta)^{\vartheta \cdot S - N}. \tag{3}$$

The probability that a number $n_1 \in \mathbb{N}^+$ of malicious nodes are assigned to `shard 1` is calculated as follows:

$$p(n_1) = \frac{\binom{N}{n_1}\binom{\vartheta \cdot S - N}{\vartheta - n_1}}{\binom{\vartheta \cdot S}{\vartheta}}. \tag{4}$$

Given Eq. (4), the conditional probability that a number $n_2 \in \mathbb{N}^+$ of malicious nodes are assigned to `shard 2` is written as follows:

$$p(n_2 \mid n_1) = \frac{\binom{N-n_1}{n_2}\binom{\vartheta \cdot S - \vartheta - (N-n_1)}{\vartheta - n_2}}{\binom{\vartheta \cdot S - \vartheta}{\vartheta}}. \tag{5}$$

Similarly, the conditional probability that a number $n_S \in \mathbb{N}^+$ of malicious nodes are assigned to `shard S` is:

$$p(n_S \mid n_1, n_2, \cdots, n_{S-1}) = \frac{\binom{N - \sum_{i=1}^{S-1} n_i}{n_S}\binom{\vartheta \cdot S - (S-1)\cdot\vartheta - \left(N - \sum_{i=1}^{S-1} n_i\right)}{\vartheta - n_S}}{\binom{\vartheta \cdot S - (S-1)\cdot\vartheta}{\vartheta}}. \tag{6}$$

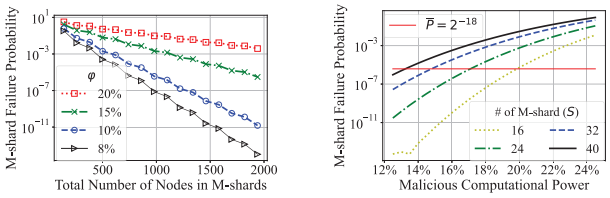
Therefore, the joint distribution of $\{n_1, n_2, \cdots, n_S\}$ is

$$p(n_1, n_2, \cdots, n_S) = p(n_1) \times p(n_2 \mid n_1) \times \cdots \times p(n_S \mid n_1, n_2, \cdots n_{S-1}). \tag{7}$$

We can then simplify Eq. (7) by referring to [29]:

$$p(n_1, n_2, \cdots, n_S) = \frac{\prod_{i=1}^{S} \binom{\vartheta}{n_i}}{\binom{\vartheta \cdot S}{N}}. \tag{8}$$

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.



(a) Failure Probability ($\overline{P}$) *vs* M-shard Size ($\vartheta \cdot S$)

(b) Failure Probability ($\overline{P}$) *vs* Malicious Computational Power (φ)

Fig. 12. Theoretical failure probability of the *M-shard*.

In a PBFT-based blockchain sharding system, as long as the proportion of malicious nodes in an M-shard is not smaller than $1/3$, the system cannot guarantee the atomicity of all intra-shard transactions. Thus, we have $\beta = \lfloor \frac{\vartheta - 1}{3} \rfloor$, and the probability that any M-shard works improperly can be represented by the real number-valued variable $\tau \in (0, 1)$:

$$\tau = 1 - p\left(n_1 \leq \beta, \cdots, n_S \leq \beta \mid X = N\right). \tag{9}$$

According to Theorem 2, we get $\delta = \varphi$. By combining Eq. (3), Eq. (8) and Eq. (9), we have the probability of M-shards failure as follows:

$$\overline{P} = \sum_{N=0}^{n \cdot S} p\left(X = N\right) \cdot \tau. \tag{10}$$

Finally, via simplifying Eq. (10), we get Eq. (2). The proof of Theorem 3 concludes. □

2) The Implications of Failure Probability of M-Shards: The failure probability of M-shards $\overline{P}$ given in Theorem 3 can help developers estimate the security level of the blockchain given system parameters ϑ, S, N and β.

Eq. (2) helps us estimate the theoretical upper bound of the probability of an M-shard's failure by referring to [5]:

$$\begin{aligned} \overline{P} &\leq \sum_{i=1}^{S} p\left(n_i > \beta\right) \\ &\leq S \cdot \sum_{i=\beta+1}^{\vartheta} \binom{\vartheta}{i} (\varphi)^i \cdot (1-\varphi)^{\vartheta - i}. \end{aligned}$$

Fig. 12(a) shows that the failure probability of M-shards (i.e., $\overline{P}$) changes with the total number of nodes in all M-shards (i.e., $\vartheta \cdot S$) when the number of shards is fixed at S=16. When the computing power of malicious nodes is high, increasing the number of nodes helps improve the security level of the system. When $\vartheta \cdot S$= 4000, Fig. 12(b) shows that the failure probability of M-shards $\overline{P}$ increases following the enlarged φ. Here, the 4 curves indicated by different configurations of S = {16, 24, 32, 40} demonstrate how $\overline{P}$ varies while the number of M-shards (i.e., S) grows. When given a fixed total number of nodes, a smaller number of shards leads to that more nodes will be assigned to each M-shard. This lowers the probability of M-shard's failure $\overline{P}$. Similar to the case with the P-shard, if the security level parameter of the M-shard is set to λ=18, i.e., $\overline{P} \leq 2^{-18}$, the system can tolerate that malicious nodes occupy approximately 20% of the overall computing power when S=16.

VII. Performance Evaluation

A. Settings

TX-driven Testbed BlockEmulator. To evaluate the proposed BrokerChain, we first developed an open-sourced experimental testbed named BlockEmulator [30], which enables the blockchain sharding mechanism. On top of BlockEmulator, we then implemented a TX-driven prototype of BrokerChain, which was implemented using Java and Python. BlockEmulator can be deployed in either local physical machines or cloud-based virtual machines such as Alibaba Cloud. Next, we ran our prototype system in BlockEmulator by replaying real-world historical TXs.

TABLE I
Symbols and Notations

T	the set of all timeslots, $t \in T$
B^t	the state block for epoch $t \in T$ at t
K	the # of brokers, $[K]$ represents the set of all brokers
S	the # of M-shards, $[S]$ represents the set of all M-shards
μ	a user of the system
$\mathbb{S}_\mu$	the account state of user μ, $\mathbb{S}_\mu = \{X_\mu \mid \mathbf{\Psi}, \eta, \omega, \zeta\}$
X_μ	the account address of user μ
$\mathbf{\Psi}$	the storage map of account, $\mathbf{\Psi} = [e_1, e_2, ..., e_S]$, e_i=0/1
η	the *nonce* field of account state
ω	the *value* field of account state
ζ	the *code* field of account state
σ_μ	the signature of user μ
Θ_{raw}	the raw cross-shard transaction
Θ_1	the first-half cross-shard TX of Θ_{raw}
Θ_2	the second-half cross-shard TX of Θ_{raw}
H	the height of block
N_{TX}	the number of transactions played back per epoch

Dataset and its Usage. We adopt two groups of real-world historical Ethereum TXs as datasets: the first dataset contains 1.67 million historical TXs recorded from Aug. 7th, 2015 to Feb. 13, 2016, while the second dataset includes 22 million transactions extracted from January to November of 2024. At the beginning of each epoch, a number N_{TX} of TXs are prepared in chronological order and replayed to the blockchain sharding system with predefined arrival rates. Those TXs are then assigned to different M-shards according to their account's state.

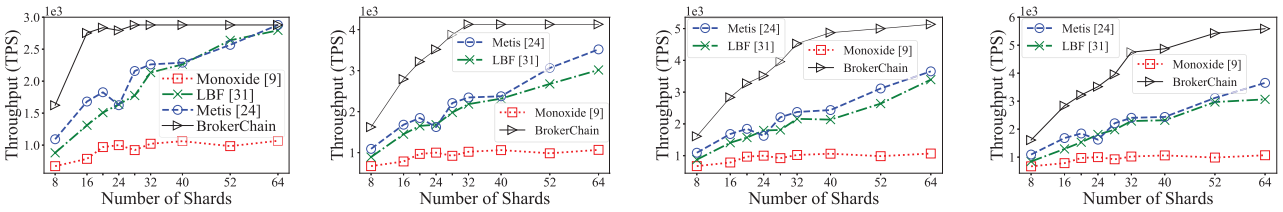
Baselines. We consider the following three baselines. Monoxide [9] distributes accounts according to the first few bits of their addresses. LBF [31] updates the distribution of accounts periodically to achieve load-balanced TX distributions. Metis [24] purely emphasizes the balanced partition on the account's state graph.

Metrics. We first study the effect of several critical system parameters considering various system metrics such as TX throughput, TX's confirmation latency, and the queue size of the TX pool. Regarding workload performance, we then measure the total and the variance of shard workloads.

B. System Throughput and Tx Confirmation Latency

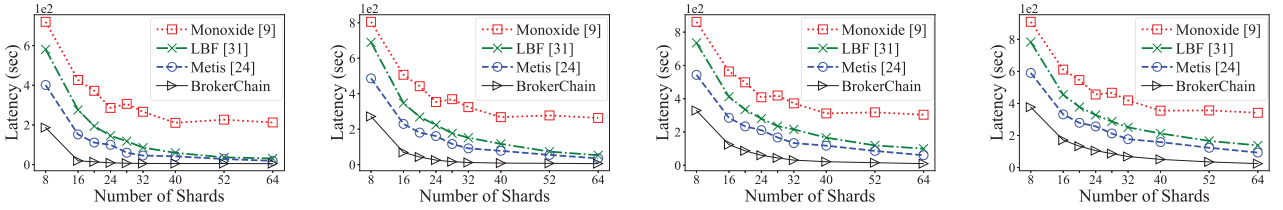
First, we evaluate throughput and transaction confirmation latency using BlockEmulator deployed in the cloud. Through renting 112 virtual machines from Alibaba Cloud, we deploy our prototype system in those virtual machines, which form 16

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.



(a) TX arrival rate=3200 TXs/Sec (b) TX arrival rate=5000 TXs/Sec (c) TX arrival rate=8000 TXs/Sec (d) TX arrival rate=16000 TXs/Sec

Fig. 13. The throughput under different methods, while varying Tx arrival rate within $\{3200, 5000, 8000, 16000\}$ TXs/Sec and increasing # of shards.



(a) TX arrival rate=3200 TXs/Sec (b) TX arrival rate=5000 TXs/Sec (c) TX arrival rate=8000 TXs/Sec (d) TX arrival rate=16000 TXs/Sec

Fig. 14. The Latency under different methods, while varying Tx arrival rate within $\{3200, 5000, 8000, 16000\}$ TXs/Sec and increasing # of shards.

TABLE II
EXPERIMENTAL RESULTS YIELDED BY CLOUD-BASED PROTOTYPE

Methods/Algorithms	Monoxide	LBF	BrokerChain
Avg. System throughput (TPS)	156.98	236.52	352.15
Avg. TX confirmation latency (sec)	1792.01	999.99	275.94

blockchain shards in total. Each virtual machine is equipped with 1 CPU core (Intel Xeon, 2.5/3.2GHz) and 2GB memory. The bandwidth of all network connections between nodes is set to 5 Mbps. For each M-shard, the block interval and block capacity are set to 8 seconds and 500 TXs, respectively. The TX arrival rate is fixed to 500 TXs/Sec. The number of broker accounts K is set to 40. Since Metis does not correlate with TX's throughput and latency, we only compare the performance of BrokerChain with Monoxide and LBF. The experimental results are shown in Table II. We see that the average throughput of BrokerChain achieves $2.24\times$ and $1.49\times$ of Monoxide's and LBF's TPS, respectively. Furthermore, BrokerChain also maintains an average TX confirmation latency of 275.94 seconds, which is much lower than that of the other two baselines.

Next, we perform TX-driven simulations in the BlockEmulator deployed in local physical machines, by tuning more sophisticated system parameters. The shard block capacity and block interval are updated to 2000 TXs and 8 seconds, respectively. K is still set to 40. Through varying the TX arrival rate and the number of shards respectively, we still study the throughput and TX's confirmation latency under different sharding mechanisms. The simulation results are shown in Fig. 13 and Fig. 14, in which the Tx arrival rate is changed within $\{3200, 5000, 8000, 16000\}$ TXs/Sec to simulate increasing workloads, and the number of shards (i.e., S) is increased from 8 to 64.

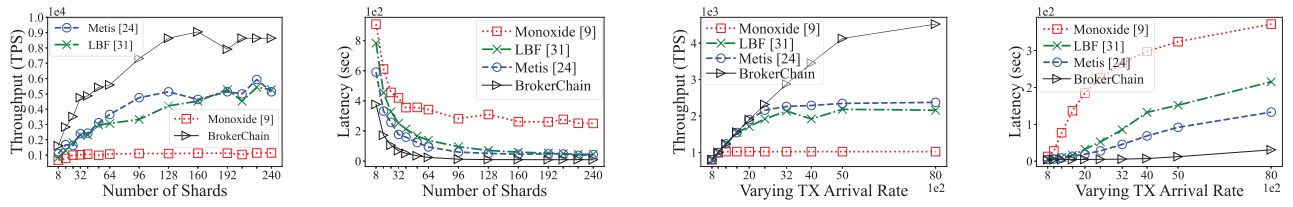
Overall, those results show that BrokerChain outperforms other baselines in terms of throughput and TX's confirmation latency. Fig. 13 demonstrates that BrokerChain exhibits a quick growth of throughput when the number of shards increases, and it converges earlier than the other three baselines when fixing a TX arrival rate. For example, when $S = 8$, all methods show a low throughput. When S grows, the throughput of all methods improves drastically. This is because a larger number of shards brings higher transaction processing capability. BrokerChain peaks its TPS at approximately 2800, 4100, and 5100 when S is set to 16, 32, and 64, and the TX arrival rate is configured with 3200, 5000, and 8000 TXs/Sec, respectively. On the other hand, the performance gap between BrokerChain and other baselines widens following the growing TX arrival rates. The reasons behind the excellent throughput of BrokerChain are two-fold: i) BrokerChain has a better workload balance across shards, and ii) it can lead to a low ratio of CTX. These two advantages help BrokerChain have a better performance that is close to the linearly increasing scalability than other baselines. We disclose more insights in the subsequent evaluations.

Fig. 14 shows the average TX confirmation latency under the same parameters as that of Fig. 13. The latency performance of all methods under each parameter setting illustrates an opposite observation compared with Fig. 13. The reasons behind those observations are also the same. Thus, we omit the description of Fig. 14.

In the previous group of evaluation, we found that the throughput of BrokerChain and the other two baselines Metis and LBF did not converge when the number of shards reached 64 and the TX arrival rate exceeded 8000 TXs/Sec. This motivates us to explore the upper-boundary performance of those methods. Thereby, we increase the number of shards while fixing the TX arrival rate at 16000 TXs/Sec. Fig. 15(a) and Fig. 15(b) show the pressure test of throughput and latency, respectively. Note that, almost all transactions are the CTXs over all shards when S reaches 128. This fact imposes a huge challenge of handling CTXs to all methods. Due to the proposed broker-based mechanism, BrokerChain exhibits excellent performance in both throughput and TX confirmation latency compared with the other three baselines. For example, BrokerChain can peak at 9048 TPS and show a low latency while increasing the number of shards. Compared with other baselines, the TPS of BrokerChain is almost $10\times$ and $2\times$ of that of Monoxide and the other two methods, respectively.

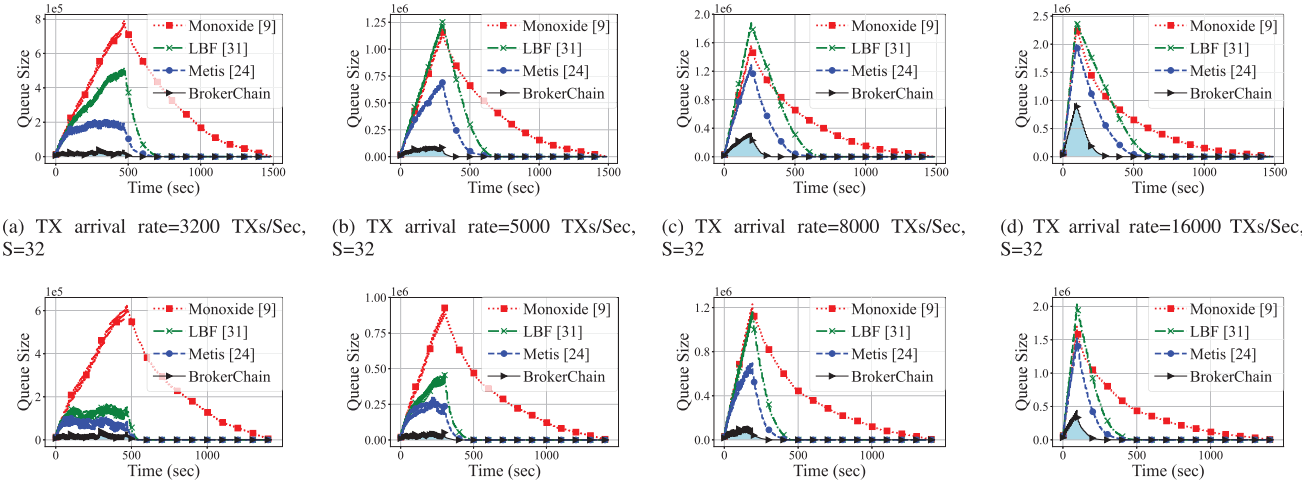
Fig. 15(c) and Fig. 15(d) show the maximum performance of throughput and latency *vs* varying TX arrival rates, while

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.



(a) TX arrival rate=16000 TXs/Sec (b) TX arrival rate=16000 TXs/Sec (c) Throughput, S=32 (d) TX confirmation latency, S=32

Fig. 15. Throughput and latency *vs* the # of shards and TX arrival rates, while maximizing system performance to its full potential.



(a) TX arrival rate=3200 TXs/Sec, S=32 (b) TX arrival rate=5000 TXs/Sec, S=32 (c) TX arrival rate=8000 TXs/Sec, S=32 (d) TX arrival rate=16000 TXs/Sec, S=32

(e) TX arrival rate=3200 TXs/Sec, S=64 (f) TX arrival rate=5000 TXs/Sec, S=64 (g) TX arrival rate=8000 TXs/Sec, S=64 (h) TX arrival rate=16000 TXs/Sec, S=64

Fig. 16. Queue size of the TX pool while changing the arrival rate of transactions.

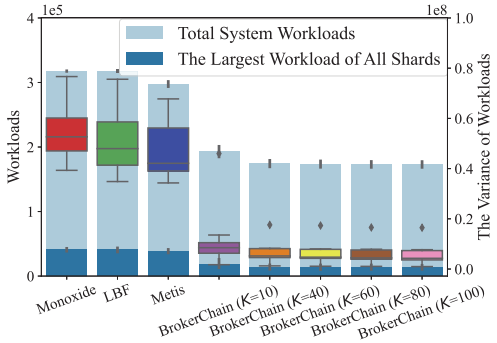


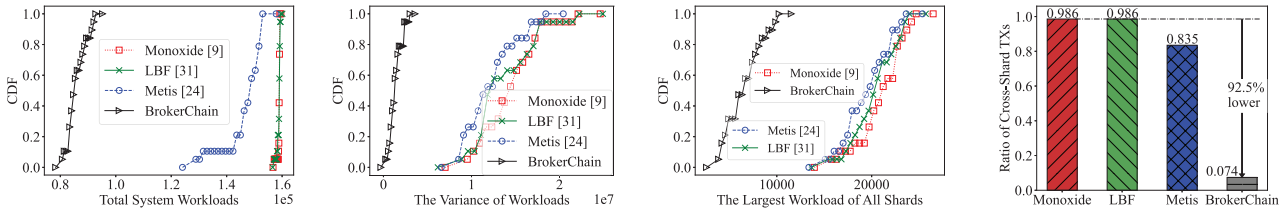
Fig. 17. Effect of the number of segmented accounts (i.e., the number of broker accounts K).

fixing $S = 32$. BrokerChain shows an overwhelming low latency compared with the other three baselines. When the TX arrival rate reaches 8000 TXs/sec, BrokerChain hits more than twice the throughput of Metis. This observation proves again that BrokerChain can guarantee low TX confirmation latency and high TPS, particularly under a high TX arrival rate.

C. Queue Size of Tx Pool

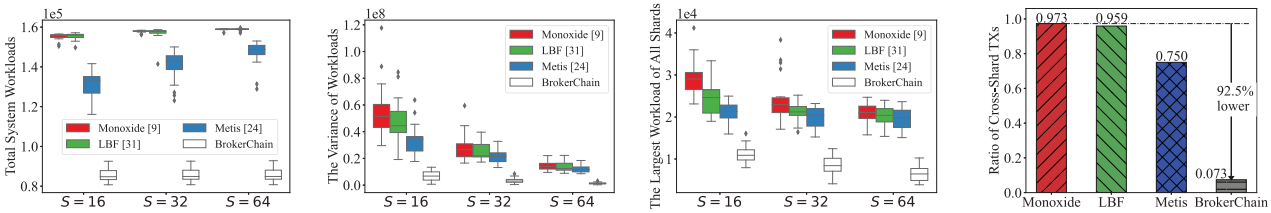
We then investigate the change of the TX pool by monitoring its queue size. We adopt a total of 1.67 million TXs to keep feeding the TX pool of the blockchain system with certain stable rates until all TXs are used up. Referring to the VISA's throughput, i.e., 4000 TPS approximately, we change the TX arrival rate from 3200 to 16000 TXs/Sec and fix $K = 40$ for BrokerChain. The number of shards is fixed to 32 or 64. Fig. 16 shows that the queue size keeps growing when injecting TXs continually at the first few hundred seconds. When all 1.67 million TXs are consumed, the queue size shrinks. Among all methods, Monoxide maintains a linearly increasing queue and shows the largest. When the TX arrival rate is lower than 5000 TXs/Sec, BrokerChain has little accumulation in the TX pool, either with 32 shards or 64 shards. Once the TX arrival rate is higher than 5000 TXs/Sec, the accumulation effect of the TX pool gradually becomes obvious. Upon comparing Fig. 16(a)–16(d) with Fig. 16(e)–16(h), a greater number of M-shards can help alleviate the transaction accumulation effect of TX pool. This result suggests that BrokerChain is more capable of maintaining a small-sized TX pool than the other three baselines. We attribute this result to the following insights. Firstly, Monoxide partitions shards based on account addresses, although accounts can be evenly distributed across shards, the transaction frequency of individual accounts is not uniform, which may lead to potential overheating in certain shards. This policy induces a large number of cross-shard TXs, which cause large processing latency. Thus, the TX pool always has a large size under Monoxide. Secondly, LBF tries to distribute TXs evenly to all shards; thus, the queue size of the TX pool is maintained at a low level under a small TX arrival rate. However, LBF is not capable of handling the cross-shard TXs timely under a large arrival rate. This is because LBF lacks consideration for cross-shard transactions, which require coordination among multiple shards. While LBF balances the workload across shards, an excessive proportion of cross-shard transactions can still impact the overall system throughput. Thirdly, Metis tries to adjust the distribution of TXs in an adaptive manner. This

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.



(a) CDF of total system workloads (b) CDF of the variance of workloads (c) CDF of the largest workload (d) Cross-shard TX ratio

Fig. 18. The workload performance under different methods, while fixing $S = 64$, $N_{\text{TX}} = 8\text{e}4$ and $K = 40$. CDF stands for cumulative distribution function.



(a) Total system workloads *vs* S (b) The variance of workloads *vs* S (c) The largest workload *vs* S (d) Cross-shard TX ratio, S=32

Fig. 19. The effect of shard # on the total, the largest and the variance of workloads, while varying S within $\{16, 32, 64\}$, and fixing $N_{\text{TX}} = 8\text{e}4$, $K = 40$.

can minimize the count of cross-shard TXs and optimize the workload balance across shards. However, the effect of Metis is not significant. This is because the transaction distribution is highly centralized, with a small number of accounts involved in the majority of transactions, making it challenging to balance the workload across shards and reduce cross-shard TXs. In contrast, BrokerChain can keep the ratio of cross-shard TXs at a low level. Thus, BrokerChain can process the accumulated transactions timely.

D. The Effect of the Number of Broker Accounts

Recall that brokers' accounts can be segmented and deployed to different shards. Now we evaluate the effect of the number of broker accounts by varying K while fixing $S = 64$, $N_{\text{TX}} = 16\text{e}4$. Through running 10 epochs of blockchain consensus, the workload performance is shown in Fig. 17, in which the total system workload is calculated as the sum of all shard's TX workloads. We see that as K increases, the total system workloads, the largest, and the variance of shard workloads all decrease. The reason is that more broker accounts can help reduce the number of cross-shard TXs and make the TX workloads more balanced than other baselines. However, when K exceeds 40, the performance-improving effect becomes saturated. This is because when K is greater than 40, the TXs involving brokers cover the majority of all TXs in the dataset, thus further increasing K does not lead to significant improvement.

E. Shard Workloads and Cross-Shard Tx Ratios

Through Fig. 18, we study the total, the largest and the variance of shard workloads under all methods. By assigning $N_{\text{TX}} = 8\text{e}4$ to $S = 64$ shards at each epoch and seting $K = 40$ for BrokerChain, Fig. 18(a) shows that BrokerChain yields the lowest total system workloads. This is because BrokerChain can reduce the number of cross-shard TXs to a certain low degree. To have a clearer insight, we compare the ratios of cross-shard TXs in Fig. 18(d). The average cross-shard TX ratios of Monoxide, LBF, Metis and BrokerChain are 98.6%, 98.6%, 83.5% and 7.4%, respectively. Furthermore, we are also curious about the breakdown of total system workloads. Fig. 18(b) and Fig. 18(c) show that BrokerChain has the smaller variances of workloads and smaller the largest workload than that of other 3 baselines. These observations imply that BrokerChain can maintain the most balanced shard workloads.

In the next group of simulation, to figure out the effect of the number of shards on workload performance, we vary S while fixing $N_{\text{TX}} = 8\text{e}4$, $K = 40$. As shown in Fig. 19(a), we see that the increasing S leads to growing total system workloads. This is because the number of cross-shard TXs increases following the increasing number of shards. Fig. 19(b) and Fig. 19(c) show that BrokerChain outperforms other baselines. Furthermore, BrokerChain also has the fewest *outliers* in figures. This observation implies that BrokerChain can make shard workloads more balanced and more stable than baselines. Again, Fig. 19(d) shows that BrokerChain still has the lowest cross-shard TX ratio.

Finally, to evaluate the effect of the number of TXs feeding per epoch on the workload performance, we vary N_{TX} within $\{8\text{e}4, 12\text{e}4, 16\text{e}4\}$ and fix $S = 64$ and $K = 40$. Fig. 20(a) shows that BrokerChain maintains around 40%-45% lower the total system workloads than baselines. Although the increasing N_{TX} leads to growing system workloads, Fig. 20(b), Fig. 20(c) and Fig. 20(d) demonstrate that BrokerChain yields the lower variance, the lower largest shard workload, and the lower cross-shard TX ratio than other baselines.

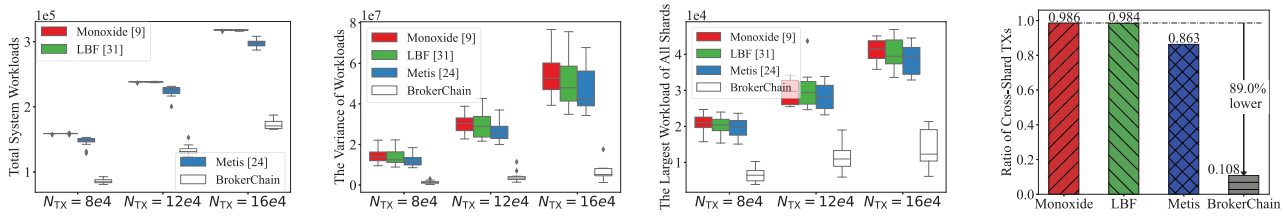
F. Visual Comparison of M-Shard Workloads

To offer a visual comparison in terms of shard workloads between BrokerChain and Monoxide, we show the heatmaps of transaction workloads under these two methods in Fig. 21. Using the same parameter settings with that of Fig. 2, 21(b) illustrates that BrokerChain can effectively eliminate hot shards and yield a much more balanced workload distribution than Monoxide.

G. The Impact of Datasets from Different Historical Periods

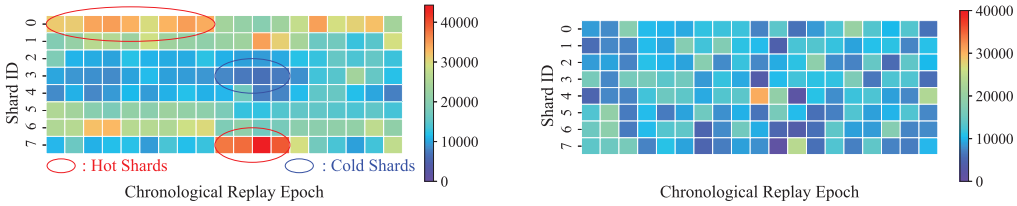
Finally, we evaluate BrokerChain's performance using two datasets collected at Ethereum's different historical periods. Except for the dataset used in previous experiments (i.e., the

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.



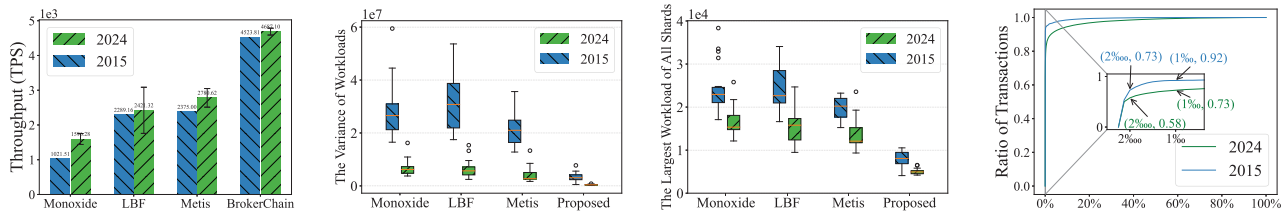
(a) Total system workloads *vs* N_{TX} (b) The variance of workloads *vs* N_{TX} (c) The largest workload *vs* N_{TX} (d) Cross-shard TX ratio, N_{TX}=12e4

Fig. 20. The effect of TX # on the total, the largest and the variance of workloads, while varying N_{TX} within {8e4, 12e4, 16e4}, and fixing $S = 64$, $K = 40$.



(a) Heatmap of transaction workload yielded by Monoxide. (b) Heatmap of transaction workload yielded by BrokerChain.

Fig. 21. The heatmap comparison of transaction workloads yielded by Monoxide and BrokerChain. Parameters are set to $S = 8$, $N_{TX} = 8e4$ for each epoch.



(a) Throughput (b) The variance of workloads (c) The largest workload (d) TX ratio *versus* accounts' rank

Fig. 22. The effect of different-stage historical datasets on throughput and workload, while using the *2015* and *2024* transaction datasets. System parameters are set to $N_{TX} = 8e4$, $S = 32$, $K = 40$. In (d), the accounts are ranked in percentage from active to inactive. Thus, (d) implies these accounts' transaction frequency.

2015 dataset), the new Ethereum transaction data adopted was generated in 2024. This new dataset is called the *2024* dataset. We extract 2 million transactions from the transactions created on the 15th of each month from January to November. Thus, the total number of transactions adopted is 22 million. We then perform 11 groups of experiments using the *2024* dataset. In each group, every 80000 transactions are injected into the BlockEmulator per epoch, i.e., N_{TX}=8e4. Other system parameters are set to S=32, K=40.

Fig. 22(a) shows that all sharding algorithms demonstrate slightly improved throughput when using the new *2024* dataset. We further compare shards' transaction workload using the two datasets *2015* and *2024*. In Fig. 22(b), the variance in shards' workload decreases for all sharding methods while using the *2024* dataset. Upon comparison, BrokerChain achieves the best in terms of workload variance. Fig. 22(c) presents the largest workload of all shards, showing that all methods benefit from the new dataset. From the new results mentioned above, we observe that using the new *2024* dataset positively influences the experimental results. This observation inspires us to think about the reason behind it. Inspired by [18], we speculate that the *transaction activity distribution* of accounts may impact the sharding blockchain's performance. Therefore, we analyze the *transaction frequency distribution* from the two datasets. As shown in Fig. 22(d), we rank all accounts by their transaction frequencies and show the ratios of associated transactions. Both datasets show that Ethereum accounts' transaction frequencies consistently follow a long-tail distribution, where a very small number of accounts are associated with extremely high transaction frequencies. For example, the top 0.02% accounts launched 58% transactions implied by the *2024* dataset. This is because, compared with the *2015* dataset, the centralization of modern transactions becomes much more weak in 2024. This shift allows those sharding methods to reduce the ratio of cross-shard transactions while distributing the transaction workloads more evenly over shards, thereby improving both sharding efficiency and the overall system performance. The new evaluation shown in this group of experiments shows that BrokerChain behaves even better when using modern transactions.

VIII. Conclusion

BrokerChain has been proposed to serve as a cross-shard protocol for the account-based sharding blockchain. In BrokerChain, the TX workload balance among all shards is achieved by the fine-grained state-graph partition and account segmentation mechanisms. BrokerChain handles the cross-shard TXs by exploiting the sophisticated broker mechanism. Compared with our conference version, this full-version article shows more design details and more theoretical analysis and presents new experimental results. The evaluation results obtained from both the cloud-based prototype and the TX-driven simulations demonstrate that BrokerChain outperforms the state-of-the-art sharding methods in terms of TX throughput, transaction confirmation latency, queue size of TX pool, and workload balance. In our future work, we plan to commercialize our BrokerChain as a real-world blockchain system that can provide an infrastructure for various decentralized applications.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.

ACKNOWLEDGMENT

The authors appreciate all the great efforts and contributions to BrokerChain made by the students who have graduated from their research laboratory, such as Jianzhou Zhan, Shenyang Zhang, Canlin Li, Yue Lin, Yetong Zhao, and other team members who are not shown in the author list, including Miaoyong Xu, Junhao Wu, and Baozhou Xie.

REFERENCES

[1] H. Huang et al., "BrokerChain: A cross-shard blockchain protocol for account/balance-based state sharding," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, London, U.K., May 2022, pp. 1968–1977.

[2] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2016, pp. 17–30.

[3] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A secure, scale-out, decentralized ledger via sharding," in *Proc. IEEE Symp. Security Privacy (SP)*, May 2018, pp. 583–598.

[4] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis, "Chainspace: A sharded smart contracts platform," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–16.

[5] M. Zamani, M. Movahedi, and M. Raykova, "RapidChain: Scaling blockchain via full sharding," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 931–948.

[6] L. N. Nguyen, T. D. T. Nguyen, T. N. Dinh, and M. T. Thai, "OptChain: Optimal transactions placement for scalable blockchain sharding," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jul. 2019, pp. 525–535.

[7] V. Bagaria, S. Kannan, D. Tse, G. Fanti, and P. Viswanath, "Prism: Deconstructing the blockchain to approach physical limits," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 585–602.

[8] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proc. Int. Conf. Manage. Data (SIGMOD)*, 2019, pp. 123–140.

[9] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *Proc. 16th USENIX Symp. Networked Syst. Design Implement. (NSDI)* 2019, Boston, MA, USA, Feb. 2019, pp. 95–112. [Online]. Available: https://www.usenix.org/conference/nsdi19/presentation/wang-jiaping

[10] Y. Tao, B. Li, J. Jiang, H. C. Ng, C. Wang, and B. Li, "On sharding open blockchains with smart contracts," in *Proc. IEEE 36th Int. Conf. Data Eng. (ICDE)*, Apr. 2020, pp. 1357–1368.

[11] H. Huang, Z. Huang, X. Peng, Z. Zheng, and S. Guo, "MVCom: Scheduling most valuable committees for the large-scale sharded blockchain," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jul. 2021, pp. 629–639.

[12] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. 3rd USENIX Symp. Operating Syst. Design Implement.*, 1999, pp. 173–186.

[13] J. Zhang, Z. Hong, X. Qiu, Y. Zhan, S. Guo, and W. Chen, "SkyChain: A deep reinforcement learning-empowered dynamic blockchain sharding system," in *Proc. 49th Int. Conf. Parallel Process. (ICPP)*, Aug. 2020, pp. 1–11.

[14] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," Bitcoin.org, White Paper, 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[15] G. Wood et al., "*Ethereum: A secure decentralised generalised transaction ledger*," Ethereum Project Yellow Paper, Ethereum Parity, Tech. Rep. SHANGHAI VERSION efc5f9a, Feb. 2025, pp. 1–42.

[16] H. Huang, W. Kong, S. Zhou, Z. Zheng, and S. Guo, "A survey of state-of-the-art on blockchains: Theories, modelings, and tools," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–42, Mar. 2021.

[17] Z. Hong, S. Guo, P. Li, and W. Chen, "Pyramid: A layered sharding blockchain system," in *Proc. Int. Conf. Comput. Commun. (INFOCOM)*, May 2021, pp. 1–10.

[18] A. Mizrahi and O. Rottenstreich, "State sharding with space-aware representations," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, May 2020, pp. 1–9.

[19] A. Mizrahi and O. Rottenstreich, "Blockchain state sharding with space-aware representations," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1571–1583, Jun. 2021.

[20] S. Zhang and J. Lee, "Double-spending with a Sybil attack in the Bitcoin decentralized network," *IEEE Trans. Ind. Informat.*, vol. 15, no. 10, pp. 5715–5722, Oct. 2019.

[21] S. Sen and M. J. Freedman, "Commensal cuckoo: Secure group partitioning for large-scale services," *ACM SIGOPS Operating Syst. Rev.*, vol. 46, no. 1, pp. 33–39, Feb. 2012.

[22] K. P. N. Puttaswamy, H. Zheng, and B. Y. Zhao, "Securing structured overlays against identity attacks," *IEEE Trans. Parallel Distrib. Syst. (TPDS)*, vol. 20, no. 10, pp. 1487–1498, Nov. 2008.

[23] B. Awerbuch and C. Scheideler, "Towards a scalable and robust DHT," *Theory Comput. Syst.*, vol. 45, no. 2, pp. 234–260, Aug. 2009.

[24] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM J. Sci. Comput.*, vol. 20, no. 1, pp. 359–392, Jan. 1998.

[25] Q. Chen, H. Huang, Z. Yin, G. Ye, and Q. Yang, "Broker2Earn: Towards maximizing broker revenue and system liquidity for sharded blockchains," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2024, pp. 251–260.

[26] D. B. Johnson, A. Menezes, and S. A. Vanstone, "The elliptic curve digital signature algorithm (ECDSA)," *Int. J. Inf. Secur.*, vol. 1, pp. 36–63, Aug. 2001.

[27] P. Maymounkov and D. Mazieres, "Kademlia: A peer-to-peer information system based on the XOR metric," in *Proc. Int. Workshop Peer-Peer Syst.*, 2002, pp. 53–65.

[28] B. Hu, C. Zhou, Y. Tian, Y. Qin, and X. Junping, "A collaborative intrusion detection approach using blockchain for multimicrogrid systems," *IEEE Trans. Syst. Man, Cybern., Syst.*, vol. 49, no. 8, pp. 1720–1730, Aug. 2019.

[29] A. Hafid, A. S. Hafid, and M. Samih, "A novel methodology-based joint hypergeometric distribution to analyze the security of sharded blockchains," *IEEE Access*, vol. 8, pp. 179389–179399, 2020.

[30] H. Huang et al., "BlockEmulator: An emulator enabling to test blockchain sharding protocols," 2023, *arXiv:2311.03612*.

[31] Z. Zhou, F. Li, H. Zhu, H. Xie, J. H. Abawajy, and M. U. Chowdhury, "An improved genetic algorithm using greedy strategy toward task scheduling optimization in cloud environments," *Neural Comput. Appl.*, vol. 32, no. 6, pp. 1531–1541, Mar. 2020.



Huawei Huang (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from The University of Aizu, Japan. He is currently an Associate Professor with the School of Software Engineering, Sun Yat-sen University, China. His research interests include blockchain systems. He has served as a Guest Editor for multiple special issues on blockchain at IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE OPEN JOURNAL OF THE COMPUTER SOCIETY, and *IET Blockchain*. He also served as a TPC Chair for a number of blockchain conferences, workshops, and symposiums like ICWS, BlockSys, and VTC Fall.



Zhaokang Yin (Graduate Student Member, IEEE) is currently pursuing the M.Sc. degree with the School of Software Engineering, Sun Yat-sen University. His research interests include blockchain.



Qinde Chen (Graduate Student Member, IEEE) is currently pursuing the Ph.D. degree with the School of Software Engineering, Sun Yat-sen University. His research interests include blockchain.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.



Jian Zheng is currently pursuing the Ph.D. degree with Sun Yat-sen University, China. His research interests include blockchain and DeFi protocols.



Xiaofei Luo received the Ph.D. degree in computer science and engineering from The University of Aizu in March 2023. He is currently a Post-Doctoral Researcher with Sun Yat-sen University. His current research interests include blockchain, payment channel networks, and consensus protocols. His research has been published in IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS and other well-known international journals and conferences.



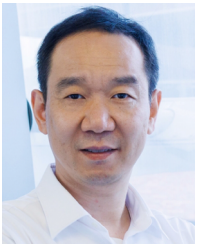
Guang Ye (Graduate Student Member, IEEE) is currently pursuing the M.Sc. degree with the School of Software Engineering, Sun Yat-sen University. His research interests include blockchain.



Xiaowen Peng is currently pursuing the M.Sc. degree with the School of Computer Science and Engineering, Sun Yat-sen University, China. His research interests focuses on blockchain.



Zibin Zheng (Fellow, IEEE) received the Ph.D. degree from The Chinese University of Hong Kong, Hong Kong, in 2012. He is currently a Professor with the School of Software Engineering, Sun Yat-sen University, China. His current research interests include service computing, blockchain, and cloud computing. He was a recipient of the Outstanding Ph.D. Dissertation Award of The Chinese University of Hong Kong in 2012, the ACM SIGSOFT Distinguished Paper Award at ICSE in 2010, the Best Student Paper Award at ICWS2010, and the IBM Ph.D. Fellowship Award in 2010. He served as a PC member for IEEE CLOUD, ICWS, SCC, ICSOC, and SOSE.



Song Guo (Fellow, IEEE) received the Ph.D. degree in computer science from the University of Ottawa. He is currently a Full Professor with the Department of Computer Science and Engineering (CSE), The Hong Kong University of Science and Technology (HKUST). Before joining HKUST in 2023, he was a Professor with The Hong Kong Polytechnic University. He has authored or co-authored over 450 papers in major conferences and journals. His current research interests include big data, cloud and edge computing, mobile computing, and distributed systems. He was a recipient of the 2019 TCBD Best Conference Paper Award, the 2018 IEEE TCGCC Best Magazine Paper Award, the 2017 IEEE Systems Journal Annual Best Paper Award, and six other Best Paper Awards from IEEE/ACM conferences. He was an IEEE Communications Society Distinguished Lecturer. He has served as an Associate Editor for IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON CLOUD COMPUTING, and IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING. He also served as the general and program chair for numerous IEEE conferences. He currently serves in the Board of Governors for the IEEE Communications Society.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:24:31 UTC from IEEE Xplore. Restrictions apply.